

Speed of Light

Scientists at The Argonne Leadership Computing Facility (ALCF) have been running calculations at a mind numbing speed of 557 trillion calculations per second for over two billion processor hours now. This brings down the research to a few weeks or months rather than years

this, you will notice that you can already see a gallery by running the project!

STEP 7: A gallery isn't as useful if you can't view the full-sized version of the image. To do functionality, we'll create a small MXML component which just displays an Image inside a panel. Right-click on your project in Flash Builder, and under the New menu, select MXML component.

In the New MXML Component dialog, enter a name for your component: `ImageViewer`. For **Based on** delete the default value, and replace it with `spark.components.TitleWindow` – it should appear as an option if you start typing `TitleWindow`. The “`TitleWindow`” component will display a window panel with shadow, a close button, and will automatically support being dragged using the title-bar. Delete the width and height values.

Inside this `TitleWindow` we will need to place our image. Add an image component as `<mx:Image id="image" />`

STEP 8: Now we will add some functionality to this component. We will define an action for closing the window, and an action to ensure that the image appears centered on the screen.

In the `<s:TitleWindow>` tag, start typing `close` as an attribute, Flash Builder will offer to auto-complete this. When you choose to auto-complete, you will get the option **Generate Close Handler** select this option and press [Enter]. It will automatically create a function which will run when the `TitleWindow`'s close action is called – the close button is pressed. The function will be called something like `titlewindow1_closeHandler`.

Inside this function, type `PopUpMan-`



A running gallery already! Right now though, you cannot view any images in their full-sized versions, which seriously hampers the usability of the gallery

ager and press [Ctrl]+Space, and press [Enter]. This will automatically include the required classes for the `PopUpManager` functionality. The `PopUpManager` handles popup windows in your application.

The code we will add to this function is:
`PopUpManager.removePopUp(this);`

This function will tell the popup manager to remove this popup window when the close button is clicked.

STEP 9: Now for centering the popup on screen. We need to center the popup on the screen when it is displayed, however it takes some time for the image to load, and that will offset the popup on the screen. So we need to center the popup when the image resizes. Just as we added the function in the previous example, add a function for the resize event for the image. After this the image tag should look like: `<mx:Image id="image" resize="image_resizeHandler(event)" />`

And we should have a function called `image_resizeHandler` for the code. In this function we will add `PopUpManager.centerPopUp(this);`

What this does is to center the popup on screen. Our image viewer code is now done.

STEP 10: Finally, we need to actually open this popup when someone clicks on an image in the list. To do this, add a function for the click event for the list. You should be left with a function called `imagesList_clickHandler`, into which we will add the following code:

```
var popupimage:ImageViewer
    = PopUpManager.createPopUp(this.root, ImageViewer, true) as ImageViewer;
popupimage.image.source =
imagesList.selectedItem.itemLargeImage;
popupimage.title = imagesList.selectedItem.itemName;
```

If you simply typed in this code, you will need to add the code to import the required classes, in this case, “`mx.managers.PopUpManager`”. Simply add “`import mx.managers.PopUpManager;`” above the `image_resizeHandler` function.

The first line of this code is using the `PopUpManager` to create a popup

window from our `ImageViewer` component. The first parameter: `this.root` tells the popup manager which display object is popping up the windows, i.e. the parent of the pop up window. The second parameter specifies what kind of window needs to be popped up, we have created our own `ImageViewer`. The third option specifies that the window is modal, i.e. it will block access to the main application while it is open. `popupimage` is the variable which is being associated with this window. With the “as `ImageViewer`” bit we are casting the windows being returned by the `PopUpManager`'s `createPopUp` function as `ImageViewer` so that we can reference it as such.

In the second line we are setting the source of the image component we added to the `ImageViewer` to the path of the large image specified by the selected item. In the third line we are setting the title of the `TitleWindow` component of our `ImageViewer` to the name of the image.

Now you'll be able to click on an image to see it in its own window. This window will appear at the center of the screen, and you will be able to drag it around. The title-bar of the window will display the name of the image, and you will be able to close this window by clicking on the close button in the title-bar.

In ten simple steps we have created a simple image gallery using Flex and Flash Builder, and we can use it quite easily with the XML galleries generated with Picasa. With Flex you can



Our gallery complete

quite easily swap out the `TileLayout` for a 3D layout or something exotic, or add functionality to select the source for the Picasa gallery XML file and load the gallery from there. 